

CHAPTER 1 트랜스포머 소개

1.1 인코더 - 디코더 프레임워크

트랜스포머 이전의 NLP에서는 순환 신경망(RNN)이 최고 수준의 성능을 보였으며, 기계 번역처럼 입력과 출력이 모두 시퀀스인 작업에는 인코더 - 디코더 구조를 사용했다.

<주요 개념>

- 순환 신경망 (RNN) = 피드백 루프를 통해 이전 스텝의 정보를 다음 스텝으로 전달하며 순차 데이터를 처리하는 신경망
- 은닉 상태 (Hidden State) = 각 스텝에서 현재 입력과 이전 정보가 누적 · 압축된 벡터
- 인코더 - 디코더 (Seq2Seq) = 인코더가 입력 시퀀스를 마지막 은닉 상태로 압축하고, 디코더가 이를 바탕으로 출력 시퀀스를 생성하는 구조

<한계>

- 정보 병목 (Information Bottleneck) = 입력 전체의 의미를 마지막 은닉 상태 하나에 담아야 해서, 시퀀스가 길수록 앞부분 정보가 손실되는 문제

1.2 어텐션 메커니즘

어텐션은 디코더가 인코더의 모든 은닉 상태에 접근하도록 하여 정보 병목 문제를 해결하는 메커니즘이다.

<주요 개념>

- 어텐션 (Attention) = 디코더가 매 타임스텝마다 인코더의 각 은닉 상태에 다른 가중치를 부여해 관련 있는 입력에 집중하는 방법
- 트랜스포머 (Transformer) = 순환 구조를 제거하고 셀프 어텐션과 피드 포워드 신경망(FF NN)만 사용하는 아키텍처
- 셀프 어텐션 (Self - Attention) = 같은 층의 모든 상태에 어텐션을 적용해, 각 단어가 문장 내 모든 단어를 동시에 참고하는 방식

<한계 및 특징>

- RNN + 어텐션의 한계 = RNN은 계산이 순차적이라 병렬 처리가 불가능하여, 훈련이 느리고 대규모로 확장하기 어려움
- 트랜스포머의 이점 = 병렬 처리가 가능해 순환 모델보다 훨씬 빠르게 훈련됨
- 발전 흐름 = RNN 인코더 - 디코더 → RNN + 어텐션 → 트랜스포머(셀프 어텐션)

1.3 NLP의 전이 학습

NLP의 전이 학습(Transfer Learning)은 대규모 텍스트 데이터로 모델을 사전 학습한 후, 학습된 지식을 특정 NLP 작업에 적용하는 방법이다. 처음부터 모델을 학습하는 방법보다 적은 양의 라벨 데이터와 적은 학습 시간으로 높은 성능을 얻을 수 있다.

<전이 학습의 주요 과정>

- 사전 학습 (Pretraining) = 대규모 비라벨 텍스트를 이용하여 다음 단어를 예측하도록 모델을 학습하는 과정

- 도메인 적응 (Domain Adaptation) = 사전 학습된 모델을 목표 데이터의 특성에 맞게 추가 학습하는 과정
- 파인튜닝 (Fine - tuning) = 특정 NLP 작업에 맞는 출력층을 추가하고 해당 작업에 맞게 모델을 학습하는 과정

<주요 전이 학습 모델>

- ULMFiT = 사전 학습된 LSTM 언어 모델을 다양한 NLP 작업에 적용할 수 있도록 제안된 전이 학습 방법 (Pretraining → Domain Adaptation → Fine - tuning)
- GPT = Transformer의 Decoder를 사용하며, 이전 단어를 바탕으로 다음 단어를 예측하는 Language Modeling 방식으로 사전 학습하는 모델
- BERT = Transformer의 Encoder를 사용하며, 문장의 일부 단어를 [MASK]로 가리고 이를 예측하는 Masked Language Modeling 방식으로 사전 학습하는 모델

1.4 허깅페이스 트랜스포머스

새로운 Transformer 모델을 실제 NLP 작업에 적용하기 위해서는 구현 및 가중치 로드, 전처리, 학습 설정 등 복잡한 작업이 필요하다. Hugging Face Transformers는 이러한 복잡한 과정을 단순화하고 다양한 모델을 통일된 방식으로 사용할 수 있도록 제공하는 라이브러리이다.

<Hugging Face Transformers의 주요 기능>

- 통일된 API = 다양한 Transformer 모델을 동일한 인터페이스를 통해 쉽게 사용할 수 있도록 지원
- 사전 학습 모델 = 이미 학습된 모델과 가중치를 쉽게 불러와 사용할 수 있도록 지원
- 다중 프레임워크 지원 = PyTorch, TensorFlow, JAX를 지원
- Task - specific Head = 텍스트 분류, 개체명 인식, 질문 답변 등 특정 작업에 맞는 출력층을 제공
- Fine - tuning = 사전 학습된 Transformer 모델을 새로운 NLP 작업에 쉽게 적용하고 학습할 수 있도록 지원

1.5 트랜스포머 애플리케이션 둘러보기

트랜스포머 아키텍처를 결합하여 해결하는 주요 자연어 처리(NLP) 과업은 다음과 같다.

<트랜스포머 주요 애플리케이션>

- 텍스트 분류 (Text Classification) = 텍스트의 문맥을 분석하여 감성 분석(긍정/부정)이나 카테고리 등을 판별하는 작업
- 개체명 인식 (Named Entity Recognition, NER) = 문장 속에서 인명, 지명, 조직명 등 고유한 의미를 가진 대상(개체)을 추출하는 작업
- 질문 답변 (Question Answering, QA) = 주어진 맥락(Context)을 바탕으로 사용자의 질문에 대한 정확한 정답을 찾아내는 작업
- 요약 (Text Summarization) = 긴 문서나 문장의 핵심 내용을 유지하면서 간결한 텍스트로 압축하는 작업
- 번역 (Translation) = 하나의 언어로 된 문장을 다른 언어의 문장으로 자동 변환하는 작업
- 텍스트 생성 (Text Generation) = 프롬프트(입력 문장)를 기반으로 문맥에 이어질 자연스러운 다음 텍스트를 자동으로 만들어내는 작업

1.6 ~ 1.8 허깅페이스 생태계 및 결론

허깅페이스는 가치를 공유하고 오픈소스 소프트웨어를 결합하여 시너지를 내는 강력한 도구 체계를 지니고 있다.

<허깅페이스 핵심 구조>

- 허깅페이스 (Hugging Face) = Hub와 여러 오픈소스 라이브러리의 유기적인 결합체
- Hub = 모델, 데이터셋, 평가 자료 등을 안전하게 저장하고 커뮤니티와 공유하는 공간
- 라이브러리 = Hub의 자원을 파이썬 코드에서 불러오고 처리 및 학습 실행을 가능하게 하는 도구

<Hugging Face에서 사용 가능한 핵심 라이브러리>

- Tokenizers = 원시 텍스트를 모델이 이해할 수 있도록 빠르게 토큰화하는 도구
- Transformers = 사전 학습된 Transformer 모델을 불러오고 학습 및 추론을 진행하는 도구
- Datasets = 대규모 데이터셋을 효율적으로 불러오고 가공, 저장, 평가에 활용하는 도구
- Accelerate = 작성한 학습 코드를 다양한 하드웨어 환경에 변경 없이 간편하게 옮겨 실행하도록 돕는 도구

<NLP의 주요 도전 과제 및 병목>

- 언어 = 영어 중심의 개발 환경으로 인한 다국어 지원 한계
- 데이터 가용성 = 특정 도메인이나 작업에 필요한 고품질 라벨링 데이터의 부족
- 긴 문서 처리 = 셀프 어텐션의 연산 복잡도로 인해 처리 가능한 문장 길이에 제한이 있음
- 불투명성 = 모델의 크기가 커짐에 따라 내부 작동 원리를 해석하기 어려운 블랙박스 문제
- 편향 = 사전 학습 데이터에 포함된 인간의 편견과 도덕적 결함이 모델 출력에 반영됨